

Dependency Manifest

Complete dependency inventory across backend, shared layer, infrastructure, and frontend.

1. `vip_shared` Lambda Layer

Located at `services/shared/python/vip_shared/`. Imported as `from vip_shared.* import ...` by every `api-*` Lambda.

Path	Purpose
<code>application/http.py</code>	HTTP response helpers: <code>json_response</code> , <code>parse_body</code> , <code>extract_caller</code> (decodes Cognito JWT claims from API Gateway event)
<code>domain/entities/filter_rule.py</code>	<code>FilterRule</code> value object — boolean operators, leaf comparisons, used by reconcile filters
<code>domain/entities/comm_limits.py</code>	<code>CommLimits</code> value object — communication frequency caps per patient
<code>domain/entities/schedule_spec.py</code>	<code>ScheduleSpec</code> — operator-defined schedule windows
<code>domain/services/filter_evaluator.py</code>	Evaluates <code>FilterRule</code> against a lead dict; pure logic
<code>domain/services/comm_limits_evaluator.py</code>	Applies <code>CommLimits</code> to decide if a patient can be contacted today
<code>domain/services/schedule_evaluator.py</code>	Decides if <code>ScheduleSpec</code> permits a contact at a given timestamp
<code>domain/services/segment_groups_translator.py</code>	Maps operator-friendly group labels ("Cancellation / 1st attempt") to Customer Profiles filter primitives
<code>infrastructure/persistence/outbound_campaigns_client.py</code>	Thin wrapper over <code>boto3.client("connectcampaignsv2")</code> with retries + structured errors
<code>infrastructure/persistence/customer_profiles_client.py</code>	Wrapper over <code>boto3.client("customer-profiles")</code>
<code>infrastructure/persistence/connect_client.py</code>	Wrapper over <code>boto3.client("connect")</code> (queues, contact flows)
<code>infrastructure/persistence/redis_lead_source.py</code>	Reads <code>wait_list:{team}:list</code> , applies server-side filters
<code>infrastructure/persistence/audit.py</code>	Writes audit records to <code>VipAdminAudit</code>
<code>infrastructure/persistence/cloudwatch_client.py</code>	Wrapper for <code>PutMetricData</code>
<code>infrastructure/persistence/segment_filter_config.py</code>	Loads filter templates for segment construction
<code>infrastructure/persistence/snapshot_reader.py</code>	Reads run snapshots for the exporter
<code>infrastructure/telemetry/structured_logger.py</code>	JSON logger with PHI scrubbing (<code>PHI_FIELDS = {phone, first_name, last_name, fullname, phone_number}</code>)
<code>infrastructure/telemetry/metrics_publisher.py</code>	High-level metric publishing helpers

Layer build: `infra/lib/utils/shared-layer.ts` builds the layer asset from `services/shared/python/`. Bundled with every `api-*` function during `cdk deploy`.

2. Per-Component Dependency Table

Backend (Python 3.12)

Component	boto3	redis	pydantic	StructuredLogger	Notes
api-plans	≥1.34	≥5.0	n/a	yes	Core orchestrator
api-campaigns	≥1.34	n/a	n/a	yes	CRUD only
api-metrics	≥1.34	n/a	n/a	yes	DDB + CW reads
api-profiles	≥1.34	n/a	n/a	yes	Customer Profiles
api-segments	≥1.34	≥5.0	n/a	yes	Reads Redis for estimates
shared (vip_shared)	≥1.34	≥5.0	n/a	self	Layer root

All api-* services declare their own `requirements.txt` for dev/test (`pytest` , `moto`); production dependencies are pulled in via the layer.

Infrastructure (`infra/`)

TypeScript / Node 20 + AWS CDK v2.

Package	Version	Purpose
<code>aws-cdk-lib</code>	<code>^2.x</code>	CDK constructs
<code>constructs</code>	<code>^10.x</code>	CDK base
<code>typescript</code>	<code>^5.x</code>	Build
<code>@types/node</code>	<code>^20.x</code>	Types

Frontend (`frontend/`)

Package	Version	Purpose
<code>react</code>	<code>^18.3</code>	UI
<code>react-dom</code>	<code>^18.3</code>	DOM renderer
<code>vite</code>	<code>^5.4</code>	Dev server + bundler
<code>typescript</code>	<code>^5</code>	Types
<code>@tanstack/react-query</code>	<code>^5</code>	Server state
<code>react-router-dom</code>	<code>^6</code>	Routing
<code>tailwindcss</code>	<code>^3</code>	CSS
<code>@radix-ui/react-*</code>	latest	shadcn/ui primitives (dialog, dropdown, popover, tooltip, etc.)
<code>lucide-react</code>	latest	Icons
<code>clsx + tailwind-merge</code>	latest	Class composition
<code>zod</code>	<code>^3</code>	API response validation

No frontend tests today (TD-003); no ESLint config today (TD-004).

3. Build & Packaging

api-plans `deploy.sh`

The script uses **Python's `zipfile` module** to build the deployment artifact rather than the system `zip` binary (the latter is broken on the dev host — see TD-014). Conceptually:

```
1. Run unit tests      (python -m pytest -q)
2. Build zip in /tmp   (python -c "import zipfile; ...")
3. Update Lambda code (aws lambda update-function-code --function-name vip-admin-ui-api-plans --zip-file fileb:///... --profile production)
4. Wait Active        (aws lambda wait function-updated --function-name vip-admin-ui-api-plans --profile production)
```

The shared layer is **separately** published. To roll a layer update:

```
cd services/shared
zip -r layer.zip python/
aws lambda publish-layer-version \
  --layer-name vip-shared \
  --zip-file fileb://layer.zip \
  --compatible-runtimes python3.12 \
  --profile production
# Then update each api-* function:
aws lambda update-function-configuration \
  --function-name vip-admin-ui-api-plans \
  --layers <new-layer-arn> \
  --profile production
```

Frontend

```
cd frontend
npm ci
npm run build      # vite build → dist/
aws s3 sync dist/ s3://vip-admin-ui-assets-165505826690/ --delete --profile production
aws cloudfront create-invalidation --distribution-id E3QCDJPG0LC07E --paths "/*" --profile production
```

Infrastructure

```
cd infra
npm ci
npx tsc -p .      # produces .js/.d.ts (TD-010 — keep these out of git)
AWS_PROFILE=production npx cdk synth
AWS_PROFILE=production npx cdk diff
AWS_PROFILE=production npx cdk deploy ApiPlansStack
```

4. Local Development Setup

Reproducible from a clean WSL Ubuntu / macOS host:

```

# 1. Clone
git clone <repo> vip-connect-external-campaigns
cd vip-connect-external-campaigns

# 2. Python tooling
python3.12 -m venv .env
source .env/bin/activate
pip install --upgrade pip
pip install boto3 redis pytest moto

# 3. Per-service deps
for svc in api-plans api-campaigns api-metrics api-profiles api-segments shared; do
  [ -f services/$svc/requirements.txt ] && pip install -r services/$svc/requirements.txt
  [ -f services/$svc/requirements-dev.txt ] && pip install -r services/$svc/requirements-dev.txt
done

# 4. Run tests
cd services/api-plans && python -m pytest -q && cd ../..
cd services/shared && python -m pytest -q && cd ../..

# 5. Frontend
cd frontend
npm ci
npm run dev                # localhost:5173 – points at production API via .env.local

# 6. Infra
cd ../infra
npm ci
npx tsc -p . --noEmit      # type-check only

```

Recommended .env.local for frontend

```

VITE_API_BASE_URL=https://<api-gateway-or-cloudfront>/api
VITE_COGNITO_USER_POOL_ID=us-east-1_MeEkW04P4
VITE_COGNITO_CLIENT_ID=<app-client-id>

```

AWS profile

```

[profile production]
sso_session = vip
sso_account_id = 165505826690
sso_role_name = DeveloperAccess
region = us-east-1

```

Always log in before deploys:

```
aws sso login --profile production
```

5. Test Inventory (262 total)

Suite	Test count	Notes
services/api-plans	150	

Suite	Test count	Notes
		Executor + store + builders + handlers
services/api-campaigns	16	CRUD layer
services/api-metrics	10	Aggregation paths
services/api-segments	23	Segment estimate + reconcile
services/api-profiles	9	Customer Profiles wrappers
services/shared	54	Filter evaluator, schedule evaluator, audit, logger
Total	262	All boto3 stubbed; no AWS calls

Run everything:

```
for d in services/api-plans services/api-campaigns services/api-metrics services/api-profiles
services/api-segments services/shared; do
  (cd "$d" && python -m pytest -q) || exit 1
done
```